

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/
<html><head> <title>307 Temporary Redirect</title> </head><body> <h1>Temporary
Redirect</h1> <p>The document has moved <a href="https://br.mirrors.cicku.me/ctan/macros/late
</body></html>
```

Relatório de Análise

GLSim — Global-Local Similarity para Reconhecimento Fino de Imagens com Vision Transformers

Artigo: [Global-Local Similarity for Efficient
Fine-Grained Image Recognition with ViT](#)

Publicado: ISCAS 2025

Autores: Edwin Arkel Rios, Min-Chun Hu, Bo-Cheng Lai
(NYCU / NTHU, Taiwan)

Data: 06/05/2026

Contents

1	Visão Geral	4
2	Estrutura de Diretórios	4
3	Arquitetura do Modelo	5
3.1	Componentes Principais (<code>glsim/model_utils/glsim.py</code>)	5
3.1.1	(a) Patch Embedding + Tokenização	5
3.1.2	(b) Encoder Transformer	5
3.1.3	(c) Módulo GLSCM — Global-Local Similarity Crop Module . . .	6
3.1.4	(d) Aggregator	6
3.1.5	(e) Cabeça de Classificação	6
3.2	Fluxo de Dados Completo	6
3.3	Variantes de Backbone Suportadas	7
4	Configuração do Método GLSim	7
5	Datasets Suportados	8
6	Pipeline de Treinamento	8
6.1	Augmentações (<code>configs/augs/</code>)	8
6.2	Otimizador e Scheduler	8
6.3	Funções de Perda Disponíveis	9
6.4	Técnicas de Regularização	9
6.5	Treinamento Distribuído	9
6.6	Experimentos e Monitoramento	9
7	Baselines Incluídos para Comparação	10
8	Ferramentas Utilitárias (<code>tools/</code>)	10
9	Dependências Principais	10
10	Uso como Módulo Python	11
11	Comandos Principais	11
11.1	Instalação	11
11.2	Treinamento (CUB, ViT-B/16, 224px)	11
11.3	Avaliação de Checkpoint	12
11.4	Inferência em Imagem Única	12
11.5	Visualização do Mecanismo Discriminativo	12
12	Resultados Reportados no Paper	12
13	Pontos Técnicos de Destaque	12

1 Visão Geral

O GLSim é um framework de reconhecimento fino de imagens (*Fine-Grained Visual Recognition*—FGVR) baseado em Vision Transformers (ViT). O problema central do FGVR é distinguir subclasses visualmente muito parecidas (ex.: espécies de pássaros, modelos de carros, cultivares de plantas), o que exige que o modelo foque em regiões locais discriminativas da imagem.

A contribuição principal é uma métrica chamada **GLS (Global-Local Similarity)**, computacionalmente barata, que identifica regiões discriminativas comparando a similaridade entre a representação global da imagem (token CLS) e as representações locais (patches). Com base nisso, a imagem é recortada, re-encodada e as *features* são fundidas por um módulo agregador.

2 Estrutura de Diretórios

Código 1: Estrutura de diretórios do repositório GLSim

```
GLSim-main/
|-- glsim/                                # Pacote principal
|   |-- model_utils/                      # Arquiteturas de modelos
|   |   |-- glsim.py                     # Modelo principal ViTGLSim
|   |   |-- configs.py                   # Configuracoes de todas as variantes de ViT
|   |   |-- build_model.py               # Factory de modelos
|   |   |-- transformer.py               # Blocos Transformer customizados
|   |   |-- timm_glsim.py                 # Integracao com timm (DINOv2, etc.)
|   |   |-- transfg.py                   # Baseline: TransFG
|   |   |-- ffvt.py                      # Baseline: FFVT
|   |   |-- cal.py                       # Baseline: CAL
|   |   |-- resnet.py                    # Baseline: ResNet
|   |-- data_utils/                       # Pipeline de dados
|   |   |-- build_dataloaders.py
|   |   |-- build_transform.py
|   |   |-- datasets.py
|   |   |-- augmentations.py
|   |-- train_utils/                      # Loop de treinamento
|   |   |-- trainer.py                   # Classe Trainer principal
|   |   |-- scheduler.py                 # Schedulers de learning rate
|   |   |-- focal_loss.py
|   |   |-- contrastive_loss.py
|   |   |-- mix.py                       # CutMix / MixUp
|   |   |-- scaler.py                    # AMP (mixed precision)
|   |-- other_utils/
|   |   |-- build_args.py                 # Parsing de argumentos CLI
|   |   |-- yaml_config_hook.py
|-- tools/                                # Scripts executaveis
|   |-- train.py                          # Treinamento e avaliacao
|   |-- inference.py                      # Inferencia em imagens individuais
|   |-- vis_dfsm.py                       # Visualizacao do mecanismo discriminativo
```

```
| |-- heatmap.py
| |-- calc_flops.py
| |-- compute_feature_metrics.py
| |-- demo.py # Demo Gradio
| '-- preprocess/ # Scripts de preparacao de datasets
|-- configs/ # Arquivos YAML de configuracao
| |-- methods/glsim.yaml # Config do metodo GLSim
| |-- datasets/*.yaml # Config de cada dataset
| |-- augs/*.yaml # Config de augmentacoes
| '-- settings/ft_is224.yaml # Config base de fine-tuning
|-- data/ # CSVs de splits (train/val/test)
|-- samples/ # Imagens de exemplo
|-- assets/ # Figuras do paper
|-- requirements.txt
'-- setup.py
```

3 Arquitetura do Modelo

3.1 Componentes Principais (glsim/model_utils/glsim.py)

A classe central é `ViTGLSim(nn.Module)`, composta por quatro módulos em sequência:

3.1.1 (a) Patch Embedding + Tokenização

Código 2: Definição do patch embedding

```
1 self.patch_embedding = nn.Conv2d(in_channels=3, out_channels
   =768,
2                                   kernel_size=(16, 16), stride
   =(16, 16))
```

- Imagem (B, 3, 224, 224) → patches (B, 196, 768) (para ViT-B/16)
- Token CLS aprendível concatenado: sequência fica (B, 197, 768)
- *Positional embedding* somado

3.1.2 (b) Encoder Transformer

Implementado em `glsim/model_utils/transformer.py`:

- `num_layers=12, hidden_size=768, num_heads=12, ff_dim=3072` (ViT-B/16)
- Cada bloco: LayerNorm → Multi-head Self-Attention → Residual → LayerNorm → FFN → Residual
- Suporte a **Stochastic Depth (DropPath)** para regularização

- Retorna *features* intermediárias e mapas de atenção para visualização

3.1.3 (c) Módulo GLSCM — Global-Local Similarity Crop Module

Código 3: Lógica do GLSCM (pseudocódigo)

```

1 get_crops(x, images):
2     g = x[:, 0, :]          # token CLS (rep. global), shape: (B,
                             # 1, 768)
3     l = x[:, 1:, :]        # patches restantes (rep. locais),
                             # shape: (B, 196, 768)
4     sim = cosine_similarity(g, l)  # similaridade por patch:
                             # (B, 196)
5     # top-k patches mais similares -> bounding box 2D ->
     # recorte + upsample

```

Intuição: Os patches mais similares ao CLS são os mais representativos do conteúdo global—logo, as regiões discriminativas da subclasse. O recorte dessas regiões é redimensionado para o tamanho original e re-encodado.

Alternativas de métrica de similaridade (configurável via `sim_metric`):

- `cos` (padrão): similaridade de cosseno, máximos selecionados
- `11 / 12`: distância, mínimos selecionados

3.1.4 (d) Aggregator

- Transformer leve com `aggregator_num_hidden_layers=1` (padrão)
- Entrada: `[CLS_original, CLS_crop]` → shape `(B, 2, 768)`
- Combina as duas representações para a predição final
- Seguido de LayerNorm

3.1.5 (e) Cabeça de Classificação

`nn.Linear(768, num_classes)` aplicado no token CLS do aggregator.

3.2 Fluxo de Dados Completo

Código 4: Pipeline completo de dados do GLSim

```

Imagem (B, 3, 224, 224)
|
v patchify_tokenize
Tokens (B, 197, 768)

```

```

    |
    v forward_encoder (ViT)
Features (B, 197, 768)
    |
    +-----+
    |         | get_crops -> recorte (B, 3, 224, 224)
    |         v patchify_tokenize + forward_encoder
    | Features Crop (B, 197, 768)
    |         |
    +---->cat (B, 394, 768)
    |
    v forward_reducer (extraí CLS de cada metade)
[CLS_orig, CLS_crop] (B, 2, 768)
    |
    v aggregator (Transformer 1 camada)
Features agregadas (B, 2, 768)
    |
    v head (Linear)
Logits (B, num_classes)

```

3.3 Variantes de Backbone Suportadas

Table 1: Variantes de backbone ViT disponíveis no GLSim

Variante	hidden_size	heads	layers	patch
ViT-T/16	192	3	12	16×16
ViT-S/16	384	6	12	16×16
ViT-B/16	768	12	12	16×16
ViT-L/16	1024	16	24	16×16
ViT-H/14	1280	16	32	14×14
DINOv2 (via timm)	768	12	12	14×14

4 Configuração do Método GLSim

Arquivo: configs/methods/glsim.yaml

Código 5: Configuração principal do método GLSim

```

classifier: 'cls'           # usa token CLS como representacao
dynamic_anchor: True       # tamanho do recorte = tamanho de um
                           patch
anchor_class_token: True   # token CLS separado para o recorte
reducer: 'cls'             # extraí apenas o token CLS de cada
                           stream

```



```
aggregator: True          # habilita o modulo aggregator
aggregator_norm: True     # LayerNorm apos aggregator
```

O modo `dynamic_anchor=True` é a configuração principal: o *bounding box* do recorte é determinado automaticamente pelo GLSCM a cada *forward pass*, adaptando-se ao conteúdo da imagem.

5 Datasets Suportados

Table 2: Datasets suportados pelo GLSim

Dataset	Domínio	Classes
CUB-200-2011	Aves	200
Stanford Cars	Veículos	196
FGVC Aircraft	Aeronaves	100
NABirds	Aves (América do Norte)	555
iNat17	Natureza geral	5.089
DAFB	Personagens anime	~2.000
Oxford Pets	Animais domésticos	37
Oxford Flowers	Flores	102
Stanford Dogs	Cães	120
Food-101	Alimentos	101
Cotton / Soy	Plantas agrícolas	varia
VegFru	Vegetais/frutas	varia
ImageNet	Geral	1.000

6 Pipeline de Treinamento

6.1 Augmentações (configs/augs/)

- **wekaugs**: flip horizontal, normalização padrão
- **medaugs**: flip + RandAugment leve
- **strongaugs**: TrivialAugmentWide, Random Erasing

6.2 Otimizador e Scheduler

- Otimizador: SGD (via `timm.optim.create_optimizer`)
- Scheduler: cosine decay com warmup (`glsim/train_utils/scheduler.py`)

- Learning rate padrão: 0.01
- Suporte a AMP (fp16) via `torch.cuda.amp.autocast + NativeScaler`

6.3 Funções de Perda Disponíveis

- `CrossEntropyLoss` (padrão)
- `LabelSmoothingCrossEntropy` (`--ls`)
- `FocalLoss` (`--focal_gamma`)
- Contrastive Loss (`contrastive_loss.py`) para TransFG

6.4 Técnicas de Regularização

- Label Smoothing
- CutMix / MixUp (`--cm, --mu`)
- Stochastic Depth (`--sd`)
- Gradient Accumulation (`--gradient_accumulation_steps`)
- Gradient Clipping (`--clip_grad`)

6.5 Treinamento Distribuído

- Suporte a `DistributedDataParallel` (DDP)
- `RASampler` para Repeated Augmentation em treino distribuído

6.6 Experimentos e Monitoramento

- Integração nativa com **Weights & Biases (WandB)**
- Salva modelo *best*, *last* e por época configurável
- Log de acurácia top-1 e top-5, loss, LR a cada `log_freq` iterações

Table 3: Modelos baseline disponíveis no repositório

Modelo	Arquivo	Referência
TransFG	<code>transfg.py</code>	Trans Fine-Grained
FFVT	<code>ffvt.py</code>	Feature Fusion ViT
CAL	<code>cal.py</code>	Counterfactual Attention Learning
ResNet	<code>resnet.py</code>	ResNet-50/101
ViT puro (sem GLSim)	<code>build_model.py</code>	—

Table 4: Scripts utilitários em `tools/`

Script	Função
<code>train.py</code>	Treinamento completo e avaliação no test set
<code>inference.py</code>	Inferência em imagem(ns) única(s); salva original + recorte lado a lado
<code>vis_dfsm.py</code>	Visualiza o mecanismo discriminativo (GLS vs. attention rollout) em lote
<code>heatmap.py</code>	Gera heatmaps de atenção
<code>calc_flops.py</code>	Calcula FLOPs e parâmetros do modelo
<code>compute_feature_metrics.py</code>	Métricas de features (separabilidade, etc.)
<code>demo.py</code>	Interface Gradio interativa
<code>preprocess/</code>	Scripts para preparar splits CSV de cada dataset

7 Baselines Incluídos para Comparação

8 Ferramentas Utilitárias (`tools/`)

9 Dependências Principais

Table 5: Principais bibliotecas utilizadas pelo GLSim

Biblioteca	Versão	Uso
PyTorch	—	framework base
timm	0.9.12	modelos pré-treinados, otimizadores, losses
einops	—	operações tensoriais legíveis
wandb	—	rastreamento de experimentos
gradio	—	demo interativa
scipy	1.8.0	utilitários científicos

10 Uso como Módulo Python

Código 6: Exemplo de uso do ViTGLSim como módulo Python

```
1 import torch
2 from glsim.model_utils import ViTGLSim, ViTConfig
3
4 cfg = ViTConfig(
5     model_name='vit_b16',
6     debugging=True,
7     classifier='cls',
8     dynamic_anchor=True,
9     reducer='cls',
10    aggregator=True,
11    aggregator_norm=True
12)
13 model = ViTGLSim(cfg)
14
15 x = torch.rand(2, 3, 224, 224)  # batch de 2 imagens RGB 224
    x224
16 out = model(x)  # retorna (logits, crops) quando anchor_size
    ativo
```

11 Comandos Principais

11.1 Instalação

Código 7: Instalação do pacote

```
pip install -e .
```

11.2 Treinamento (CUB, ViT-B/16, 224px)

Código 8: Treinamento no dataset CUB-200-2011 com ViT-B/16

```
python tools/train.py \
    --cfg configs/cub_ft_is224_medaugs.yaml \
    --lr 0.01 \
    --model_name vit_b16 \
    --cfg_method configs/methods/glsim.yaml
```

11.3 Avaliação de Checkpoint

Código 9: Avaliação de um checkpoint salvo

```
python tools/train.py --ckpt_path ckpts/cub_glsim_224.pth --
test_only
```

11.4 Inferência em Imagem Única

Código 10: Inferência com visualização do recorte discriminativo

```
python tools/inference.py \
--ckpt_path ckpts/dafb_glsim.pth \
--images_path samples/others/dafb_rena_170785.jpg \
--vis_mask glsim_norm
```

11.5 Visualização do Mecanismo Discriminativo

Código 11: Visualização do GLSCM em lote (NABirds + DINOv2)

```
python tools/vis_dfsm.py \
--serial 52 --batch_size 4 --vis_cols 4 \
--cfg configs/nabirds_ft_is224_weakaugs.yaml \
--model_name glsvit_base_patch14_dinov2.lvd142m \
--vis_mask gls --vis_mask_pow
```

12 Resultados Reportados no Paper

O método obtém resultados competitivos em múltiplos benchmarks de FGVR. Os destaques são:

- **CUB-200-2011:** Acurácia top-1 comparável ao estado da arte em 224px e 448px.
- **NABirds / iNat17:** Vantagem em datasets de grande escala.
- **Eficiência:** O GLSCM tem custo computacional ordens de magnitude menor que *attention rollout*, com FLOPs totais similares ao ViT base.
- Checkpoints pré-treinados disponíveis no HuggingFace: [NYCU-PCSxNTHU-MIS/GLSim](#).

13 Pontos Técnicos de Destaque

1. **Sem necessidade de anotações de localização:** O GLSCM detecta regiões discriminativas automaticamente, sem *bounding boxes* supervisionados.

2. **Custo baixo do GLSCM:** A similaridade cosseno entre o CLS e os patches é uma operação $\mathcal{O}(N)$ sobre a sequência de patches — muito mais barata que *attention rollout* ($\mathcal{O}(N^2)$ por camada).
3. **Dynamic anchor:** O tamanho do recorte é determinado dinamicamente pelos índices dos top- k patches, adaptando-se ao tamanho do objeto em cada imagem.
4. **Modularidade:** O código separa claramente backbone, mecanismo de crop, agregator e head. É possível usar o ViTGLSim diretamente como módulo ou via `build_model` com qualquer backbone do timm.
5. **Test-time augmentation:** Suporte a flip horizontal no momento de avaliação (`test_flip`), que melhora marginalmente a acurácia.

14 Limitações Identificadas no Código

- `F.upsample_bilinear` está deprecado no PyTorch moderno (deveria ser `F.interpolate(..., mode='bilinear')`).
- O trainer usa `torch.cuda.synchronize()` no loop de treino mesmo sem necessidade em modo não-distribuído, adicionando overhead desnecessário.
- A função `imshow` para visualização de erros chama `input()` interativo, incompatível com ambientes headless/servidor.
- `setup.py` usa o padrão legado em vez de `pyproject.toml`.